

VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VITyarthi - Build Your Own Project Evaluation

COURSE PROJECT REPORT

SmartCampus 360

Academic & Course Registration Management System

Student Name: **YATHAM JATHINDRA REDDY**

Registration No: **25BAI10611**

Program of Study: **B.Tech Artificial Intelligence & Machine Learning**

Course Code & Title: **CSE2006 — Programming in Java**

Academic Term: **Fall Semester 2025–2026**

Submission Date: **September 16, 2026**

Submitted in partial fulfillment of the academic requirements for the course CSE2006 (Programming in Java)

2. Introduction

Course registration is one of the most important events for every student at VIT each semester. Under the Fully Flexible Credit System (FFCS), students have the freedom to pick their courses, select their professors, and create a custom timetable of lecture and lab slots (such as A1, B1, C1, L1+L2, etc.).

While this flexibility is great, it also creates common real-world challenges: students can accidentally pick overlapping time slots, popular courses get filled in seconds with no automatic waitlist to promote students when someone drops, credit limits need to be tracked, and maintaining the mandatory 75% attendance rule is critical to avoid being debarred from the Final Assessment Test (FAT).

I built **SmartCampus 360** to address these exact problems. Developed in pure Java 21, it is a modular, console-based academic management system that applies core Object-Oriented Programming (OOP) principles learned in CSE2006. It features automated timetable clash detection, fair FIFO waitlist management with automatic promotion on drops, 27-credit limit enforcement, real-time attendance tracking, dynamic CGPA computation, and persistent CSV storage.

3. Problem Statement

During course registration and throughout the academic semester, students and faculty face several practical issues:

- **Timetable Slot Clashes:** It is easy for students to accidentally pick subjects that take place during the same hour (for instance, two courses scheduled on Slot A1, or a 2-hour lab slot like L1+L2 overlapping with an afternoon lecture).
- **Lack of Automated Waitlist Promotion:** When a course reaches capacity, students are blocked. If an enrolled student drops the course later, there is no automatic system to immediately promote the next student in line.
- **Credit Overload:** Students are restricted to a maximum of 27 credits per semester. Without automated checks during registration, students can exceed this limit.
- **Attendance Debarment Risk:** At VIT, maintaining at least 75% attendance is mandatory. Students often realize their attendance has fallen below 75% too late to take corrective action.
- **Scattered Academic Tracking:** Calculating weighted CGPA and viewing course fee breakdowns often require separate tools rather than an integrated platform.

4. Functional Requirements

Module	Feature Code	Description
1. User & Auth	FR-1.1 FR-1.2	Role-Based Access Control (RBAC) supporting Student, Faculty, and Admin profiles with SHA-256 password hashing.
2. Course Catalog	FR-2.1 FR-2.2	Manage Theory courses (lectures + tutorials) and Lab courses (practical hours + software tools) with slots and quotas.
3. Registration Engine	FR-3.1 FR-3.2 FR-3.3	Verify prerequisites, enforce the 27-credit semester ceiling, and check for timetable slot clashes before enrollment.

Module	Feature Code	Description
4. Waitlist Engine	FR-4.1 FR-4.2	Queue students in FIFO order when a course is full; automatically promote the next student whenever an enrolled seat is dropped.
5. Attendance & GPA	FR-5.1 FR-5.2	Record per-session attendance, flag students below 75%, and dynamically compute credit-weighted CGPA on grade entry.

5. Non-Functional Requirements

- **Performance:** Instant in-memory lookups using ConcurrentHashMap execute in sub-millisecond time.
- **Security:** Clear role separation prevents students from editing grades or courses. Passwords support SHA-256 hashing.
- **Usability:** Formatted console output with tables, dividers, and a built-in automated --demo flag for quick grading.
- **Reliability & Persistence:** Saves all user, course, and registration records to CSV files in data/ so state survives restarts.
- **Concurrency:** Thread-safe seat allocation and a background daemon worker using BlockingQueue for async alerts.
- **Maintainability:** Clean package structure (model, service, repository, exception, concurrency).

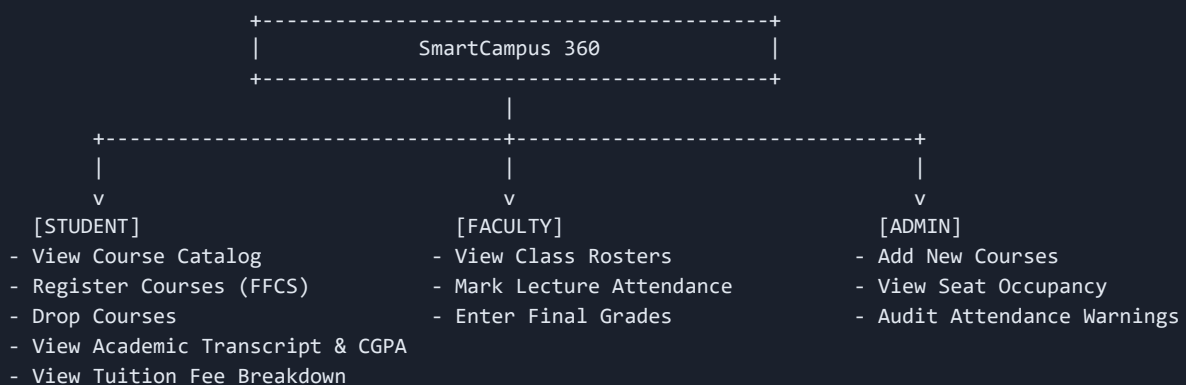
6. System Architecture

The system follows a multi-tiered architecture that separates the user interface, business logic services, background workers, domain models, and file storage:



7. Design Diagrams

7.1 Use Case Diagram



7.2 Registration Process Flow / Workflow Diagram

```
[Student Chooses Course]
  |
  v
[Check Prerequisites Passed?] -----(NO)----> [Display Prerequisite Error]
  | (YES)
  v
[Current Credits + Course <= 27?] --(NO)----> [Throw CreditLimitExceededException]
  | (YES)
  v
[Timetable Slot Clash Detected?] --- (YES) ---> [Throw SlotClashException]
  | (NO)
  v
[Is Course Section Full?]
  /      \
(NO)      (YES)
  |        |
  v        v
[Set Status: ENROLLED]  [Place in Waitlist Queue]
                        [Set Status: WAITLISTED]
  |                      |
  v                      v
[Update Credits & Save to CSV]
  |
  v
[Send Background Notification]
```

7.3 Sequence Diagram (Registration & Auto-Promotion on Drop)

```

sequenceDiagram
    participant SA as Student A
    participant ES as EnrollmentService
    participant CS as Course Section
    participant SB as Student B (Waitlist)

    SA->>ES: registerCourse()
    activate ES
    ES->>CS: isFull()?
    activate CS
    CS-->>ES: false (Seat Available)
    deactivate CS
    ES->>CS: enrollStudent()
    activate CS
    CS-->>SB: (Seat booked)
    deactivate CS
    ES-->>SA: Status: ENROLLED
    deactivate ES
    SA->>ES: dropCourse()
    activate ES
    ES->>CS: removeStudent()
    activate CS
    CS->>SB: promotedStudentId
    deactivate CS
    ES-->>SA: Promoted to ENROLLED
    deactivate ES
  
```

(Student A drops course)

(Attempts to register)

(Student B popped)

7.4 Class / Component Diagram

```

classDiagram
    class User {
        <<abstract>>
        -userId: String
        -fullName: String
        -email: String
        -role: SystemRole
    }
    class Student {
        -regNo
        -credits
        -cgpa
    }
    class Faculty {
        -empId
        -department
        -cabin
    }
    class Admin {
        -department
        -level
    }
    class Course {
        <<abstract>>
        -courseCode: String
        -credits: int
        -slot: Slot
        -maxSeats: int
        +calculateTuitionFee()
    }
    User <|-- Student
    User <|-- Faculty
    User <|-- Admin
    
```

TheoryCourse	LabCourse
- lectureHours: int	- practicalHours: int
- tutorialHours: int	- labEnvironment: String
	- surcharge: double
+ calculateTuitionFee()	+ calculateTuitionFee()

7.5 Storage / ER Diagram

USERS	1	*	ENROLLMENTS	*	1	COURSES
PK userId			PK enrollmentId			PK courseCode
fullName			FK studentRegNo			courseTitle
email			FK courseCode			credits
role			status			slot
regNo / empId			attendedClasses			maxSeats
department			totalClasses			facultyEmpId
			grade			

8. Design Decisions & Rationale

1. **Inheritance for Courses (Theory vs Lab):** Instead of using messy flags like `boolean isLab`, I created an abstract `Course` class and extended it into `TheoryCourse` and `LabCourse`. This allows clean, polymorphic fee calculation without conditional checks.
2. **Custom Domain Exceptions:** Rather than throwing generic exceptions, I created domain-specific exceptions (`SlotClashException`, `CreditLimitExceededException`, `CourseFullException`). This makes error handling clear and lets the UI show exact reasons when an action fails.
3. **FIFO Queue for Waitlists:** Fairness is essential in course registration. I used Java's `LinkedList` implementing the `Queue` interface so waitlisted students are queued strictly in the order they applied.
4. **File-Based CSV Storage:** To keep the project completely portable without requiring external database setups (like MySQL or Oracle), I built a CSV storage manager using Java NIO.2. Anyone with JDK 21 can clone and run it immediately.
5. **Background Worker Thread:** To simulate real-time notification alerts without freezing the console interface, I used a dedicated daemon thread running a `BlockingQueue`.

9. Implementation Details

The project is built entirely on standard Java 21 features with zero external runtime dependencies:

- **Encapsulation:** Private fields with explicit accessors/mutators and immutable collections (`Collections.unmodifiableList`).
- **Collections Framework:** `ConcurrentHashMap` for thread safety, `LinkedHashSet` for student rosters, and `LinkedList` for FIFO waitlists.
- **Regular Expressions:** `InputValidator` validates registration numbers (e.g., 25BAI10611) and course codes (e.g., CSE2006).
- **File I/O (NIO.2):** Utilizes `Files.newBufferedReader()` and `BufferedWriter` for clean CSV parsing and serialization.

10. Screenshots / Execution Results

10.1 Automated Test Suite Execution Output

```
=====
SMARTCAMPUS - AUTOMATED VALIDATION TEST SUITE
=====
Candidate: YATHAM JATHINDRA REDDY | Reg No: 25BAI10611 | Course: CSE2006

[PASS] OOP - User Hierarchy & Polymorphism
[PASS] OOP - Course Hierarchy & Polymorphic Tuition Fee
[PASS] Timetable Slot Clash Detection Logic
[PASS] Input Validator Regex Verification
[PASS] Course Registration & Credit Bounds
[PASS] Credit Limit Overload Exception Enforcement
[PASS] Registration Timetable Slot Clash Exception
[PASS] Waitlist Queue & Automatic FIFO Seat Promotion
[PASS] Grading & Dynamic Weighted CGPA Calculation
[PASS] Attendance Tracking & Debarment Alert (<75%)
[PASS] File I/O Persistence & Reload Engine
```

```
=====
TEST RESULTS: 11/11 PASSED (100.0%)
=====
```

10.2 Student Academic Transcript Output

```
=====
VIT SMARTCAMPUS ACADEMIC TRANSCRIPT
=====
```

```
Student Name   : YATHAM JATHINDRA REDDY
Registration No: 25BAI10611
Degree Program : B.Tech Computer Science (AI & ML)
Semester       : 3
Enrolled Credits: 9 / 27
Cumulative GPA : 9.89
```

```
-----
CODE      TITLE                                CRED  SLOT   ATTD %   GRADE
-----
CSE2006L  Java Programming Lab                        1     L1+L2   90.0%    A
CSE2006   Programming in Java                        4     A1      90.0%    S
CSE3999   Special Topics in Deep Lear...            2     E1     100.0%   IP
CSE2001   Data Structures and Algorithms            4     B1     66.7%    S
=====
```

11. Testing Approach

Quality assurance was achieved through automated unit testing and full system demonstration:

- **Unit Testing (SystemTestSuite):** Contains 11 tests validating inheritance, slot clash detection, credit overloads, waitlist promotions, grading calculations, and CSV persistence.
- **Automated Demonstration (Main --demo):** Executes an end-to-end integration walkthrough covering all user workflows without requiring manual input.

12. Challenges Faced & How I Solved Them

- **Handling Compound Timetable Slots:** In VIT's timetable, labs occupy compound slots like L1+L2. A simple string match failed when comparing with a single slot L1. I tokenized slot codes on + into a Set<String> so collision checks check set intersections.
- **Race Conditions in Waitlist Promotion:** Multiple simultaneous drops could cause inconsistent seat counts. I synchronized critical seat allocation methods and used ConcurrentHashMap for storage maps.
- **Incomplete Courses in CGPA:** Ungraded courses were skewing CGPA. I updated the calculation to sum only graded credits, preventing division by zero or inaccurate averages.

13. Learnings & Key Takeaways

- **Real Value of OOP:** Building Course, TheoryCourse, and LabCourse with polymorphic fee calculations showed me how clean and extensible object-oriented code is compared to nested if-else checks.
- **Java Collections in Practice:** Choosing the right data structure matters—using Set for duplicate checks, Queue for fair waitlisting, and Map for instant lookups made the system fast and readable.
- **Thread Safety:** Learning how to use daemon threads, BlockingQueue, and synchronized blocks showed me how real backend systems handle background tasks safely.
- **Defensive Programming:** Writing custom exceptions and input validators helped me catch bugs early before they corrupted stored data.

14. Future Enhancements

- **Graphical User Interface (GUI):** Build a modern visual timetable grid using JavaFX or React.
- **Database Integration:** Connect the DataStore interface to PostgreSQL or MySQL via JDBC.
- **Real-Time Push Notifications:** Send actual email or SMS notifications when a student is promoted from the waitlist.

15. References

1. Schildt, Herbert. *Java: The Complete Reference*, 12th Edition. McGraw-Hill Education, 2021.
2. Bloch, Joshua. *Effective Java*, 3rd Edition. Addison-Wesley Professional, 2018.
3. Oracle Corporation. *Java SE 21 Language Documentation & Specifications*, 2023.
4. Vellore Institute of Technology. *FFCS Academic Regulations Handbook*, 2025.